

ADDA-CUDA

CUDA acceleration of [ADDA](#), the light-scattering simulator based on the discrete dipole approximation (DDA).

This repository contains a CUDA port developed from the official ADDA source tree, while preserving the ADDA calculation pipeline and command-line interface.

ADDA calculates electromagnetic scattering and absorption by particles of arbitrary shape and composition. It can also handle particles near a plane substrate and calculate emission or decay-rate enhancement. See the [official ADDA project](#) and the [ADDA manual](#) for the physical model and complete list of options.

1 Features

- CPU and NVIDIA CUDA implementations of the ADDA solver.
- Single precision (`float32`) and double precision (`float64`) executables.
- Full 3-D GPU FFT implementation.
- Slice-based GPU FFT implementation for reduced VRAM consumption.
- Low-memory CUDA organization for larger problems.
- CUDA-resident iterative solver vectors, avoiding CPU–GPU transfers during the QMR calculation.
- CUDA implementations of the available iterative-solver vector operations.
- Windows build based on Makefiles, MinGW, GCC, G++, and GFortran.
- Native Linux build with GCC, FFTW, CMake, and NVIDIA CUDA.
- Separate CUDA backend compilation with `nvcc`, using the Windows `.bat` or Linux `.sh` script.
- Three-level circulant preconditioners developed from Steven Lanier’s work, including `lanier_full`, `lanier_partition`, `lanier_nested`, and `lanier_multizone`. The `ai/` directory contains `lanier_tqc_v1_ema_actor.bin`, the trained actor file created by Steven Lanier and used to configure the parameters of the Lanier-preconditioned solver. Reference: [Steven Lanier, “Learning to precondition: Reinforcement learning enhanced three-level circulant preconditioning for the Discrete Dipole Approximation,” JQSRT 350 \(2026\), 109741](#). Scientific details are provided in [adda_cuda_extensions.pdf](#), [adda_cuda_extensions.tex](#), and [adda_cuda_extensions.md](#).
- Development provenance: the main code was developed from the sequential ADDA repository code, without MPI and without OpenCL, with assistance from ChatGPT Sol 5.6 in web mode. The CUDA port started from upstream ADDA version `1.5.0-alpha3`, branch `master`, commit [8f550a7786bd4cff5abf3cb2f2180690a08e1157](#) (short SHA `8f550a7`), authored by Maxim Yurkin and dated 30 March 2026 at 08:45:28 UTC, with commit message “minor changes”. This exact upstream commit is the reference baseline for source-code diffs against ADDA-CUDA. The Lanier preconditioner code was developed from Steven Lanier’s article and from written code supplied by Steven Lanier.

2 Split CUDA backend and CLion host debugging

The default architecture is controlled by:

```
option(ADDA_CUDA_SPLIT_BACKEND
    "Use kernel.cu with the CMake-compiled C++ wrapper backend"
    ON)
```

With `ADDA_CUDA_SPLIT_BACKEND=ON`, the backend is separated into three parts. `src/kernel.cu` contains only CUDA `__global__` kernels, required `__device__` helpers, and C-linkage launch wrappers. It is the only source compiled by `nvcc`. `src/kernel.h` defines the private C ABI used by those launch wrappers. `src/wrappermatvec_backend.cpp` contains the host-side CUDA state, allocations, cuFFT/cuBLAS orchestration, MatVec dispatch, iterative-solver and Lanier backend logic, and all public functions declared by `src/cudamatvec_backend.h`. CMake compiles this file with GNU `g++`.

The historical `src/cudamatvec_backend.cu` is preserved unchanged. When `ADDA_CUDA_SPLIT_BACKEND=OFF`, CMake uses the historical monolithic backend and the existing libraries under `cuda-backend/debug/` or `cuda-backend/release/`.

The public backend ABI is unchanged. The split kernel source is built once for double precision and once with `ADDA_CUDA_SINGLE_BACKEND` for single precision. Both kernel libraries are stored in `cuda-backend/kernels/`. There is intentionally no separate Debug kernel variant: Debug and Release CMake profiles share the same `nvcc`-built kernel library, while the host wrapper is compiled separately with the normal `g++` Debug or Release flags.

On native Windows, `nvcc` uses a supported MSVC host compiler for `kernel.cu`, while CLion/CMake compile `wrappermatvec_backend.cpp` with MinGW `g++`. On Linux and WSL, `nvcc` builds `kernel.cu` and the selected Linux/WSL GNU `g++` compiles the wrapper. This allows ordinary source-level breakpoints and single stepping in the CUDA host code from CLion.

3 Build scripts and generated files

With the default split backend, the canonical scripts `scripts/build_cuda_backends.bat` and `scripts/build_cuda_backends.sh` compile only `src/kernel.cu`. They produce one double-precision and one single-precision kernel library in `cuda-backend/kernels/`. The single-precision build defines `ADDA_CUDA_SINGLE_BACKEND`. The same kernel binaries are used by Debug and Release CMake profiles.

Platform	Precision	Kernel runtime	Link input
Windows	double	<code>cuda-backend/kernels/adda_cuda_kernels.dll</code>	<code>libadda_cuda_kernels.a</code>
Windows	single	<code>cuda-backend/kernels/adda_cuda_kernels_single.dll</code>	<code>libadda_cuda_kernels_single.a</code>
Linux/WSL	double	<code>cuda-backend/kernels/libadda_cuda_kernels.so</code>	same <code>.so</code>
Linux/WSL	single	<code>cuda-backend/kernels/libadda_cuda_kernels_single.so</code>	same <code>.so</code>

CMake then builds `adda_cuda_backend` and `adda_cuda_backend_single` from `src/wrappermatvec_backend.cpp` with GNU `g++`. The historical Debug/Release backend directories are retained for `ADDA_CUDA_SPLIT_BACKEND=OFF`.

The complete build scripts then create self-contained output directories:

Script	Final directory	Files collected there
scripts/build_all.bat	build_windows/bin/	Eight .exe files, CUDA DLLs, FFTW DLLs, available MinGW runtime DLLs, and ai/
scripts/build_all.sh	build_linux/bin/	Eight Linux executables, both CUDA .so files, and ai/

build_all.bat accepts an optional CUDA architecture and MinGW bin directory. build_all.sh accepts an optional CUDA architecture and number of parallel jobs. When the job count is omitted, each script detects the number of processors available on the current machine.

4 Executables

The complete scripts produce the following executables in build_windows/bin/ or build_linux/bin/.

Executable	Processor	Precision	FFT / memory organization	Purpose
adda.exe	CPU	double / float64	Standard FFT	Reference ADDA implementation
adda_single.exe	CPU	single / float32	Standard FFT	CPU single-precision comparison
adda_cuda.exe	NVIDIA GPU	double / float64	Full 3-D GPU FFT	Accurate CUDA version; highest VRAM use
adda_cuda_single.exe	NVIDIA GPU	single / float32	Full 3-D GPU FFT	Lower-memory and faster CUDA version
adda_cuda_slice.exe	NVIDIA GPU	double / float64	Slice FFT	Reduced VRAM in double precision
adda_single_slice.exe	NVIDIA GPU	single / float32	Slice FFT	Lowest-memory slice variant
adda_low_mem.exe	NVIDIA GPU	single / float32	Low-memory layout	Minimum large resident GPU buffers
adda_low_mem_double.exe	NVIDIA GPU	double / float64	Low-memory layout	Low-memory layout with double precision

The CUDA backend DLLs are:

DLL	Precision	Role
adda_cuda_backend.dll	double / float64	CUDA backend for double-precision CUDA executables
adda_cuda_backend_single.dll	single / float32	CUDA backend for single-precision CUDA executables

On Linux/WSL with the split backend, the CMake-built host wrapper libraries are emitted beside the executables and depend on the kernel libraries from cuda-backend/kernels/. With ADDA_CUDA_SPLIT_BACKEND=OFF, the historical libraries in cuda-backend/debug/ or cuda-backend/release/ are used instead. The Linux executables have the same names as in the table above, without the .exe suffix.

5 CMake entry points

The root CMakeLists.txt contains the shared source lists and build rules. The platform-specific files only select the appropriate options and call the root configuration, so the source list is not duplicated:

CMakeLists.txt	Shared CMake configuration
windows/CMakeLists.txt	Windows / MinGW / CUDA entry point
linux/CMakeLists.txt	Linux / CPU and CUDA entry point

Use the platform-specific entry point when configuring from a clean checkout. Do not configure the Windows build from `linux/`, or the Linux build from `windows/`.

Complete Windows build with MinGW:

<pre>scripts\build_all.bat 70 "F:\mingw64_11.2\bin"</pre>

Complete Linux build for `sm_70`:

<pre>bash scripts/build_all.sh 70</pre>

The values `70` and `sm_70` both select CUDA compute capability 7.0. It is the default target used in the commands documented above; use the value matching the target GPU when another architecture is required. If the architecture argument is omitted, the scripts pass `native` to `nvcc` so it can detect the GPU installed on the build machine.

On Windows, `"F:/mingw64_11.2/bin"` is the MinGW `bin` directory containing `gcc.exe`, `g++.exe`, and `gfortran.exe`. GFortran is required by the default complete build. This second argument is optional. When it is omitted, `build_all.bat` finds `gcc.exe` from the Windows `PATH` and uses the directory containing it.

The root `CMakeLists.txt` remains the shared implementation used by both entry points.

6 CLion build

The project combines three groups of source files:

- C and C++ compilation units and include fragments: `src/*.c`, `src/*.cpp`, and `src/*.inc`;
- Fortran sources, enabled by default: `src/fort/*.f` and `src/fort/*.f90`;
- CUDA sources: `src/*.cu`.

The C sources require the C99 standard, so the Microsoft C/C++ compiler (MSVC) cannot be used to compile the ADDA C code. On Windows, however, the CUDA compiler `nvcc` uses a supported MSVC toolset as its host compiler. CLion builds the C, C++, and Fortran sources with the GNU compilers `gcc`, `g++`, and `gfortran`, using a MinGW, Linux/System, or WSL toolchain.

For the default split backend, run the kernel build script before configuring the CUDA targets in CLion:

<pre>scripts\build_cuda_backends.bat</pre>
--

or, on Linux/WSL:

<pre>bash scripts/build_cuda_backends.sh</pre>
--

These scripts compile only `src/kernel.cu`; no dedicated CUDA Debug variant is required. CMake compiles `src/wrappermatvec_backend.cpp` with the GNU C++ compiler selected by the CLion toolchain. A Debug profile therefore provides normal `g++` symbols for the CUDA host code, including allocations, `cuFFT`/`cuBLAS` orchestration, `MatVec` dispatch, iterative solver operations, and Lanier logic. Kernel source debugging remains a separate CUDA-debugger task. Set `-DADDA_CUDA_SPLIT_BACKEND=OFF` only when intentionally selecting the preserved historical monolithic backend.

Native Windows builds produce `.exe` files. Linux and WSL builds produce Linux ELF executables, which must be run from Linux or a WSL console. Do not configure `linux/CMakeLists.txt` with MinGW, or `windows/CMakeLists.txt` with Linux/WSL.

7 Command line build

7.1 Linux

Run the complete script from the repository root. The optional first argument is the GPU compute capability without the decimal point (70 for `sm_70`, 86 for `sm_86`, and so on). The optional second argument is the number of parallel jobs:

```
bash scripts/build_all.sh 70
```

This command creates Debug and Release two precision-specific `.so` files in `cuda-backend/debug/` and `cuda-backend/release/`, configures a Release build in `build_linux/`, builds all eight programs, then copies the shared libraries and the complete `ai/` directory into `build_linux/bin/`.

The final directory contains:

```
build_linux/bin/  
|-- adda  
|-- adda_single  
|-- adda_cuda  
|-- adda_cuda_single  
|-- adda_cuda_slice  
|-- adda_single_slice  
|-- adda_low_mem  
|-- adda_low_mem_double  
|-- libadda_cuda_backend.so  
|-- libadda_cuda_backend_single.so  
'-- ai/
```

A short CPU single-precision smoke test is:

```
./build_linux/bin/adda_single -grid 8 8 8 -maxiter 20
```

The Linux configuration deliberately uses the system FFTW libraries rather than the Windows libraries stored in `fftw/`. Custom native FFTW paths can be provided with `FFTW3_ROOT`, `FFTW3_LIBRARY`, and `FFTW3F_LIBRARY`.

7.2 Windows

From a Windows command prompt, run:

```
scripts\build_all.bat 70 "F:\mingw64_11.2\bin"
```

The CUDA architecture and MinGW directory are optional. The script first calls `build_cuda_backends.bat`, configures CMake in `build_windows/`, builds all eight executables, and collects the runtime files in `build_windows/bin/`.

The resulting directory has the following structure:

```
build_windows/  
|-- CMakeCache.txt  
|-- CMakeFiles/  
|-- Makefile or build.ninja  
|-- cmake_install.cmake  
'-- bin/  
    |-- adda.exe  
    |-- adda_single.exe  
    |-- adda_cuda.exe  
    |-- adda_cuda_single.exe  
    |-- adda_cuda_slice.exe  
    |-- adda_single_slice.exe  
    |-- adda_low_mem.exe  
    |-- adda_low_mem_double.exe
```

```
|-- adda_cuda_backend.dll
|-- adda_cuda_backend_single.dll
|-- libfftw3-3.dll
|-- libfftw3f-3.dll
|-- available MinGW runtime DLLs
'-- ai/
```

The CMake cache, generated build system, and object files remain under `build_windows/`. Only `build_windows/bin/` is intended as the runtime directory. The CUDA import libraries remain in `cuda-backend/`; only the CUDA DLLs are copied beside the executables.

To rebuild only the CUDA libraries and their import libraries, run:

```
scripts\build_cuda_backends.bat 70 "F:\mingw64_11.2\bin"
```

This backend-only command writes Debug and Release double- and single-precision DLLs, MSVC `.lib` files, and MinGW `.a` import libraries to `cuda-backend/debug/` and `cuda-backend/release/`. It does not rebuild the eight ADDA executables.

8 Requirements

8.1 Linux

- C and Fortran compilers supported by the project (GCC and GFortran are the tested configuration).
- CMake 3.20 or newer and either Make or Ninja.
- FFTW development libraries for both precisions. On Debian/Ubuntu, install `libfftw3-dev`.
- For the CUDA targets: an NVIDIA CUDA Toolkit providing `nvcc`, `cuFFT`, and `cuBLAS`. A compatible NVIDIA GPU and driver are required to run the CUDA executables.

For example, on Debian/Ubuntu:

```
sudo apt update
sudo apt install build-essential gfortran cmake libfftw3-dev
```

Install the NVIDIA CUDA Toolkit separately if `nvcc--version` is not available. The distribution package name and supported CUDA version depend on the Linux and NVIDIA driver versions.

8.2 Windows

- 64-bit Windows.
- MinGW-w64 with GCC, GFortran, and `dlltool`.
- CMake and either MinGW Makefiles or Ninja.
- NVIDIA GPU and a compatible NVIDIA driver.
- NVIDIA CUDA Toolkit with `nvcc` and a supported MSVC host compiler (`cl.exe`).
- The FFTW headers, import libraries, and runtime DLLs stored in `fftw/`.

The root `CMakeLists.txt` builds the ADDA C and Fortran sources by default. `scripts/build_all.bat` follows this CMake setting: it does not override `ADDA_NO_FORTRAN`, and puts the selected MinGW `bin` directory first in `PATH` so CMake can discover the matching GNU compilers. It also copies the available GFortran runtime DLLs beside the executables when they are present. To disable Fortran, change the default in `CMakeLists.txt` or configure with `-DADDA_NO_FORTRAN=ON`. The CUDA `.cu` source remains compiled separately by `nvcc`, which uses the supported MSVC host compiler on Windows.

9 The `fftw/` directory

The `fftw/` directory is not included in this GitHub repository. Before the standard Windows build, the user must download or copy a compatible Windows FFTW3 distribution into the repository root as `fftw/`. CMake detects this directory automatically when `fftw/fftw3.h` is present. Alternatively, an existing FFTW installation can be selected explicitly with the CMake variables shown below.

The files most useful for building ADDA on Windows are:

File	Purpose
<code>fftw3.h</code>	FFTW3 C header included by ADDA
<code>libfftw3.dll.a</code>	MinGW import library for double precision
<code>libfftw3-3.dll</code>	Runtime DLL for double-precision CPU/CUDA executables
<code>libfftw3f.dll.a</code>	MinGW import library for single precision
<code>libfftw3f-3.dll</code>	Runtime DLL for single-precision executables
<code>libfftw3.a</code> , <code>libfftw3f.a</code>	Static libraries, available for alternative link configurations

The directory also contains FFTW Fortran interfaces, long-double/quadruple precision variants, threaded/OpenMP variants, and FFTW wisdom utilities. They are not required by the default ADDA CMake targets. The CMake build copies the appropriate `libfftw3-3.dll` or `libfftw3f-3.dll` next to each executable.

For a different FFTW installation, override the automatic detection, for example:

```
cmake -S windows -B build_windows -G Ninja '
-DFFTW3_ROOT="C:/path/to/fftw" '
-DFFTW3_LIBRARY="C:/path/to/fftw/libfftw3.dll.a" '
-DFFTW3F_LIBRARY="C:/path/to/fftw/libfftw3f.dll.a"
```

On Linux, the files in this Windows-oriented `fftw/` directory should not be used as native libraries. Install FFTW for Linux or provide Linux-compatible paths through `FFTW3_ROOT`, `FFTW3_LIBRARY`, and `FFTW3F_LIBRARY`.

The names above describe the builds tested on Windows on 28 August 2026. The repository contains source code and build scripts; compiled binaries are not required to be committed to Git.

10 Precision and convergence

Single precision reduces the storage required by complex vectors and FFT workspaces by approximately a factor of two. It is useful for problems that do not fit in GPU memory in double precision, but it also reduces the attainable numerical accuracy and can make iterative convergence more difficult.

Double precision is recommended for validation, difficult or poorly conditioned problems, and final production results. In single precision, `qmr2` is often preferable to `qmr` because it has better round-off properties. The final residual can be checked with:

```
adda_cuda_single.exe ... -iter qmr2 -recalc_resid
```

For CUDA single-precision calculations, the `-reliable_resid` option periodically evaluates the true residual instead of relying only on the recursively estimated residual. It is intended to limit residual drift and improve convergence reliability. In the current implementation it applies to CUDA float32 BiCGStab(2)/BCGS2 and GPBiCGStab(2):

```
adda_cuda_single.exe ... -iter bcgs2 -reliable_resid
adda_cuda_single.exe ... -iter gpbcgstab2 -reliable_resid
```

The true residual is checked every 20 iterations. With `-reliable_resid`, the Krylov calculation is restarted only when the discrepancy indicates a significant residual error. With `-reliable_resid_force_restart`, a restart is forced at every 20-iteration check, even when the recursively estimated residual is still considered reliable; this option implies `-reliable_resid`.

Examples:

```
adda_cuda_single.exe ... -iter bcgs2 -reliable_resid
adda_cuda_single.exe ... -iter bcgs2 -reliable_resid_force_restart
```

These options are ignored for unsupported solvers or for non-CUDA/non-single-precision builds. They can be combined with `-recalc_resid` when a final independent residual check is required.

11 Iterative solvers

Compared with the original ADDA solver set, this version adds iterative solvers also available in [IFDDA](#):

The command-line syntax is `-iter SOLVER`; the space between `-iter` and the solver name is required.

Solver	Command-line option	Alias
Bi-CGStab(2)	<code>-iter bcgs2</code>	<code>-iter bicgstab2</code>
Bi-CG	<code>-iter bicg</code>	
Bi-CGStab	<code>-iter bicgstab</code>	
CGNR	<code>-iter cgnr</code>	
CSYM	<code>-iter csym</code>	
QMR	<code>-iter qmr</code>	
QMR2	<code>-iter qmr2</code>	
BiCGStab(4)	<code>-iter bicgstab4</code>	<code>-iter bicgstabl4</code>
BiCGStab(8)	<code>-iter bicgstab8</code>	<code>-iter bicgstabl8</code>
BiCGStab(12)	<code>-iter bicgstab12</code>	<code>-iter bicgstabl12</code>
GPBiCGStab(2)	<code>-iter gpbcgstab2</code>	<code>-iter gpbcgstabl2</code>
GPBiCGStab(4)	<code>-iter gpbcgstab4</code>	<code>-iter gpbcgstabl4</code>

The default solver remains `qmr`. For single-precision calculations, `qmr2` is a useful first choice when round-off limits convergence.

12 Running ADDA

ADDA uses command-line options. For example, a 32-cubed grid and refractive index 1.6 can be requested with options of the following form:

```
adda_cuda_single.exe -grid 32 -m 1.6 0 ...
```

The complete syntax, geometry options, material definitions, solver options, and output files are documented in the [ADDA manual](#). Use `-h` for the help available in the executable.

13 Repository layout

<code>src/</code>	ADDA C sources and CUDA integration
<code>src/kernel.cu</code>	CUDA kernels and nvcc launch wrappers
<code>src/kernel.h</code>	Private C launch-wrapper ABI
<code>src/wrappermatvec_backend.cpp</code>	Host CUDA backend compiled by g++
<code>src/cudamatvec_backend.cu</code>	Preserved historical monolithic backend
<code>cuda-backend/</code>	Separate CUDA backend build files
<code>scripts/</code>	Windows and Linux CUDA build scripts
<code>tests/</code>	Regression and CUDA tests
<code>third_party/</code>	Required third-party headers
<code>doc/</code>	ADDA documentation

14 Relationship to ADDA

ADDA-CUDA is based on the source code of the official [ADDA project](#). The upstream baseline used to start the CUDA port is ADDA version 1.5.0-alpha3, branch `master`, commit [8f550a7786bd4cff5abf3cb2f2180690a08e1157](#) (short SHA 8f550a7), authored by Maxim Yurkin and dated 30 March 2026 at 08:45:28 UTC. The commit message is “minor changes”. Using this exact commit as the upstream baseline makes it possible to compare ADDA-CUDA with the original ADDA source using a Git diff. The CUDA implementation is adapted to ADDA’s C99 code, FFT pipeline, complex arithmetic, and iterative solvers.

For the original ADDA scientific references, capabilities, limitations, and licensing information, please consult the official ADDA repository and the [ADDA manual](#). This project is intended to remain compatible with the upstream ADDA command-line workflow wherever the CUDA implementation permits.

15 License and citation

ADDA is distributed under the GNU General Public License. Please consult the license and attribution information in the upstream project before redistributing modified versions.

If results obtained with `adda-cuda` are published, please cite the original ADDA work and the relevant discrete-dipole-approximation references listed in the official ADDA documentation. Please also reference the `adda-cuda` repository: <https://github.com/michelgross34/adda-cuda>

For reproducibility, please also report the ADDA-CUDA release tag or commit SHA used for the published calculations. If a Lanier preconditioner is used, please also cite Steven Lanier’s paper: “[Learning to precondition: Reinforcement learning enhanced three-level circulant preconditioning for the Discrete Dipole Approximation](#),” JQSRT 350 (2026), 109741.

16 Status

This repository is an experimental CUDA extension of ADDA. CPU and CUDA results should be compared on representative cases, especially when using single precision, slice FFT, or low-memory

modes. Numerical agreement and convergence should be validated before using a configuration for production calculations.